

BOOKMYSHOW DATABASE MANAGEMENT SYSTEM

Database Design and SQL Implementation

Developed By

Chennaboina Jayanthi

Roll No.: 2023001650

Department of Computer Science and Engineering

Gandhi Institute of Technology and

Management (GITAM)2025

1. Introduction

1.1 Overview of the project

BookMyShow is India's leading online ticketing platform for movies, concerts, plays, and sports events. Established in 2007, it connects **patrons**, **organizers**, and **venues** via a centralized system, providing real-time seat availability, secure payments, and instant ticket delivery.

This case study models BookMyShow using **DBMS concepts**, identifying entities, attributes, and relationships to design a robust database supporting bookings, payments, seat allocation, and customer feedback.

1.2 Objectives

- Identify key **entities and attributes**.
 - Define **relationships** and cardinalities.
 - Create a **conceptual ER diagram**.
 - Develop a **logical schema** with constraints (PK, FK, etc.).
 - Implement queries, views, and PL/SQL for transaction management.
 - Ensure **data consistency, integrity, and security**.
-

2. Requirement Analysis

2.1 Entities and Attributes

Entity	PK	Attributes	FK
Patron	patron_id	phone, name, email, preferred_language	loyalty_wallet_id
Organizer	org_id	name, contact, tax_details	—
Venue	venue_id	name, address, city, map_link	—
Hall	hall_id	name, capacity, hall_type	venue_id
Event	event_id	title, type, runtime_mins, language	organizer_id
Show	show_id	start_time, end_time, status, doors_open_before_mins	event_id, hall_id
Seat	seat_id	row_label, seat_no, zone, status	hall_id
PricingPlan	plan_id	zone, price, convenience_fee, valid_from, valid_to	show_id
CartHold	hold_id	created_at, expires_at, status	patron_id, show_id
Booking	booking_id	total_amount, booking_time, status	patron_id, show_id, payment_id
Payment	payment_id	amount, method, status, gateway_ref	booking_id
Ticket	ticket_id	qr_code, status, admitted_at	booking_id, seat_id
Review	review_id	rating, comment, created_at	patron_id, event_id
SupportCase	case_id	issue_type, status, opened_at	patron_id, booking_id

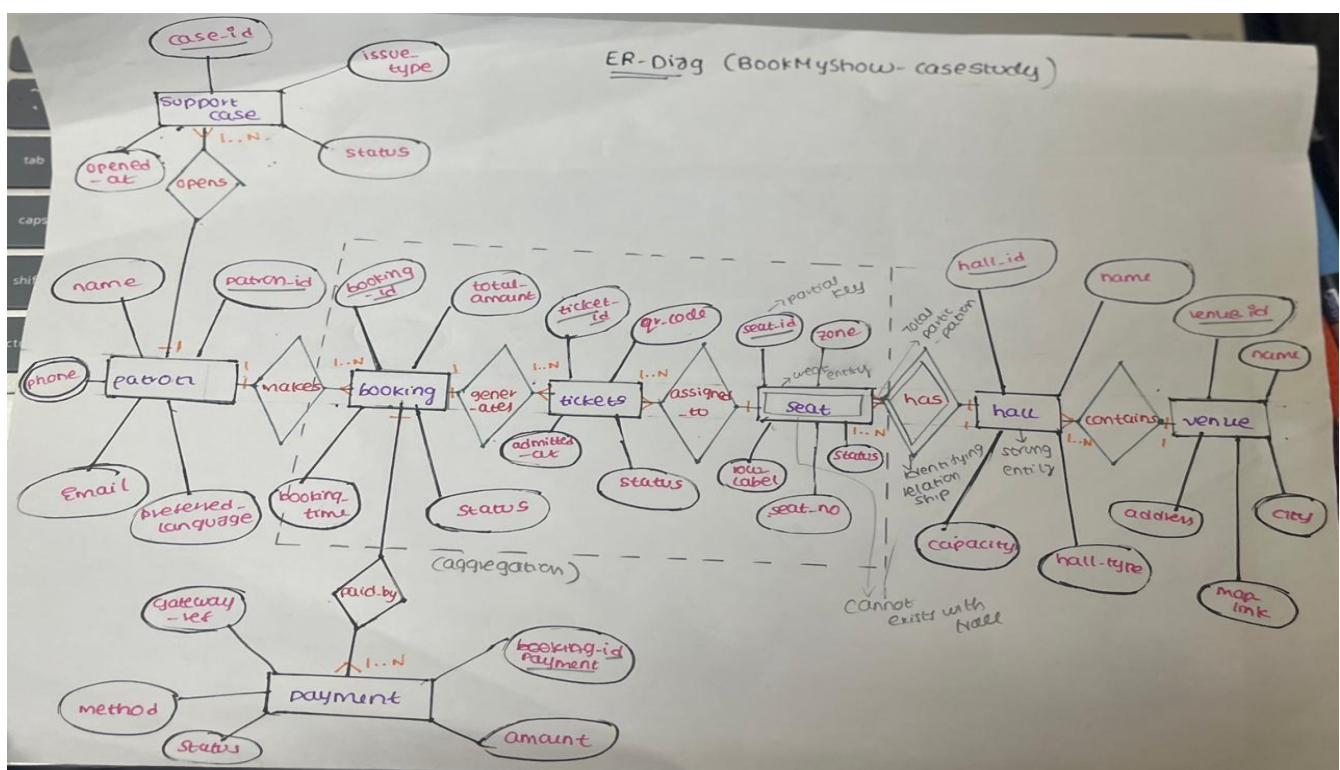
Entity	PK	Attributes	FK
LoyaltyWallet	wallet_id	balance, points	patron_id
Companion	companion_id	name, phone	patron_id

2.2 Relationships

Relationship	From	To	Cardinality	Description
Hosts	Organizer	Event	1 — M	One organizer can host multiple events
Scheduled_As	Event	Show	1 — M	One event can have multiple shows
Defines	Show	PricingPlan	1 — M	One show can have multiple pricing plans
Makes	Patron	Booking	1 — M	A patron can make multiple bookings
Generates	Booking	Ticket	1 — M	One booking generates multiple tickets
Assigned_To	Ticket	Seat	M — 1	Each ticket is for one seat
Has	Seat	Hall	M — 1	Seats belong to a hall
Contains	Hall	Venue	M — 1	Halls belong to a venue
Paid_By	Booking	Payment	1 — M	One booking can have multiple payments
Opens	Patron	SupportCase	1 — M	Patrons can open multiple support cases
Holds	Patron	CartHold	1 — M	Patron can hold multiple seat selections
For	CartHold	Show	M — 1	Each cart hold is for a show
Writes	Patron	Review	1 — M	Patron can write multiple reviews
About	Review	Event	M — 1	Review is for a specific event

3. Conceptual Design with ER Model

3.1 Entity Relationship Diagrams (ERD)



4. Logical Database Design

4.1 Conceptual Schemas (Tables with PKs, FKs, and constraints)

1. Patron

```
CREATE TABLE Patron (  
    patron_id INT PRIMARY KEY,  
    phone VARCHAR(15) NOT NULL UNIQUE,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    preferred_language VARCHAR(20),  
    loyalty_wallet_id INT UNIQUE  
);
```

Result

Table PATRON created.

Elapsed: 00:00:00.035

Sample Records (8)

```
INSERT INTO Patron VALUES  
(1, '9876543210', 'Arun', 'arun@gmail.com', 'English', 1),  
(2, '8765432109', 'Meera', 'meera@gmail.com', 'Hindi', 2),  
(3, '7654321098', 'Ravi', 'ravi@gmail.com', 'Telugu', 3),  
(4, '6543210987', 'Priya', 'priya@gmail.com', 'English', 4),  
(5, '5432109876', 'Anil', 'anil@gmail.com', 'Hindi', 5),  
(6, '4321098765', 'Sara', 'sara@gmail.com', 'English', 6),  
(7, '3210987654', 'Vikram', 'vikram@gmail.com', 'Telugu', 7),  
(8, '2109876543', 'Neha', 'neha@gmail.com', 'English', 8);
```

Download ▾ Execution time: 0.15 seconds						
	PATRON_ID	PHONE	NAME	EMAIL	PREFERRED_LANGU	LOYALTY_WALLET_
1	1	9876543210	Arun	arun@gmail.com	English	1
2	2	8765432109	Meera	meera@gmail.com	Hindi	2
3	3	7654321098	Ravi	ravi@gmail.com	Telugu	3
4	4	6543210987	Priya	priya@gmail.com	English	4
5	5	5432109876	Anil	anil@gmail.com	Hindi	5
6	6	4321098765	Sara	sara@gmail.com	English	6
7	7	3210987654	Vikram	vikram@gmail.com	Telugu	7
8	8	2109876543	Neha	neha@gmail.com	English	8

2. Organizer

```
CREATE TABLE Organizer (
```

```

org_id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
contact VARCHAR(15),
tax_details VARCHAR(50)
);

```

Result

Table ORGANIZER created.

Elapsed: 00:00:00.021



Sample Records (8)

```

INSERT INTO Organizer VALUES
(1, 'XYZ Studios', '9876543211', 'GST12345'),
(2, 'ABC Promoters', '9876543212', 'GST12346'),
(3, 'Mega Cinema', '9876543213', 'GST12347'),
(4, 'LiveNation', '9876543214', 'GST12348'),
(5, 'DreamWorks', '9876543215', 'GST12349'),
(6, 'EventCorp', '9876543216', 'GST12350'),
(7, 'CinemaPlus', '9876543217', 'GST12351'),
(8, 'StarMovies', '9876543218', 'GST12352');

```



Download ▾

Execution time: 0.014 seconds

	ORG_ID	NAME	CONTACT	TAX_DETAILS
1	1	XYZ Studios	9876543211	GST12345
2	2	ABC Promoters	9876543212	GST12346
3	3	Mega Cinema	9876543213	GST12347
4	4	LiveNation	9876543214	GST12348
5	5	DreamWorks	9876543215	GST12349
6	6	EventCorp	9876543216	GST12350
7	7	CinemaPlus	9876543217	GST12351
8	8	StarMovies	9876543218	GST12352

3. Venue

```

CREATE TABLE Venue (
venue_id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
address VARCHAR(200),
city VARCHAR(50),
map_link VARCHAR(200)
);

```

Result

Table VENUE created.

Elapsed: 00:00:00.024



Sample Records (8)

INSERT INTO Venue VALUES

(1, 'PVR Koramangala', 'Koramangala, Bangalore', 'Bangalore', 'link1'),
(2, 'INOX MG Road', 'MG Road, Bangalore', 'Bangalore', 'link2'),
(3, 'Sathyam Cinemas', 'Royapettah, Chennai', 'Chennai', 'link3'),
(4, 'Cinapolis Pune', 'Shivaji Nagar, Pune', 'Pune', 'link4'),
(5, 'Wave Cinemas', 'Hitech City, Hyderabad', 'Hyderabad', 'link5'),
(6, 'PVR Phoenix', 'Phoenix Marketcity, Mumbai', 'Mumbai', 'link6'),
(7, 'INOX Delhi', 'Connaught Place, Delhi', 'Delhi', 'link7'),
(8, 'Cinapolis Noida', 'Sector 18, Noida', 'Noida', 'link8');

	VENUE_ID	NAME	ADDRESS	CITY	MAP_LINK
1	1	PVR Koramangala	Koramangala, Banga	Bangalore	link1
2	2	INOX MG Road	MG Road, Bangalore	Bangalore	link2
3	3	Sathyam Cinemas	Royapettah, Chennai	Chennai	link3
4	4	Cinapolis Pune	Shivaji Nagar, Pune	Pune	link4
5	5	Wave Cinemas	Hitech City, Hyderabad	Hyderabad	link5
6	6	PVR Phoenix	Phoenix Marketcity, Mumbai	Mumbai	link6
7	7	INOX Delhi	Connaught Place, De	Delhi	link7
8	8	Cinapolis Noida	Sector 18, Noida	Noida	link8

4. Hall

```
CREATE TABLE Hall (  
  hall_id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  capacity INT NOT NULL,  
  hall_type VARCHAR(20),  
  venue_id INT NOT NULL,  
  FOREIGN KEY (venue_id) REFERENCES Venue(venue_id)  
);
```

Result

Table HALL created.

Elapsed: 00:00:00.027



Sample Records (8)

INSERT INTO Hall VALUES

(1, 'Hall A', 200, 'IMAX', 1),
(2, 'Hall B', 150, '2D', 1),
(3, 'Hall 1', 300, '3D', 2),
(4, 'Hall 2', 250, '2D', 2),
(5, 'Main Hall', 400, 'IMAX', 3),
(6, 'Screen 1', 180, '2D', 4),
(7, 'Screen 2', 220, '3D', 4),
(8, 'VIP Hall', 50, 'VIP', 5);

	HALL_ID	NAME	CAPACITY	HALL_TYPE	VENUE_ID
1	1	Hall A	200	IMAX	1
2	2	Hall B	150	2D	1
3	3	Hall 1	300	3D	2
4	4	Hall 2	250	2D	2
5	5	Main Hall	400	IMAX	3
6	6	Screen 1	180	2D	4
7	7	Screen 2	220	3D	4
8	8	VIP Hall	50	VIP	5

5. Event

```
CREATE TABLE Event (  
  event_id INT PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  type VARCHAR(20),  
  runtime_mins INT,  
  language VARCHAR(20),  
  organizer_id INT NOT NULL,  
  FOREIGN KEY (organizer_id) REFERENCES Organizer(org_id)  
);
```


Result

Table EVENT created.

Elapsed: 00:00:00.028

Sample Records (8)

INSERT INTO Event VALUES
(1, 'Avengers: Endgame', 'Movie', 180, 'English', 1),
(2, 'Rock Concert', 'Concert', 120, 'English', 2),
(3, 'Shakespeare Play', 'Play', 150, 'English', 3),
(4, 'Cricket Match', 'Match', 360, 'English', 4),
(5, '3D Animation Movie', 'Movie', 100, 'English', 5),
(6, 'Classical Music', 'Concert', 90, 'Hindi', 6),
(7, 'Drama Night', 'Play', 130, 'Telugu', 7),
(8, 'Football Finals', 'Match', 300, 'English', 8);

Download Execution time: 0.011 seconds

	EVENT_ID	TITLE	TYPE	RUNTIME_MINS	LANGUAGE	ORGANIZER_ID
1	1	Avengers: Endgame	Movie	180	English	1
2	2	Rock Concert	Concert	120	English	2
3	3	Shakespeare Play	Play	150	English	3
4	4	Cricket Match	Match	360	English	4
5	5	3D Animation Movie	Movie	100	English	5
6	6	Classical Music	Concert	90	Hindi	6
7	7	Drama Night	Play	130	Telugu	7
8	8	Football Finals	Match	300	English	8

6. Show

```
CREATE TABLE Show (  
  show_id NUMBER PRIMARY KEY,  
  start_time TIMESTAMP NOT NULL,  
  end_time TIMESTAMP NOT NULL,  
  status VARCHAR2(20),  
  doors_open_before_mins NUMBER,  
  event_id NUMBER NOT NULL,  
  hall_id NUMBER NOT NULL,  
  FOREIGN KEY (event_id) REFERENCES Event(event_id),  
  FOREIGN KEY (hall_id) REFERENCES Hall(hall_id),  
  CHECK (end_time > start_time);
```

Result

Table SHOW created.

Elapsed: 00:00:00.021



Sample Records (8)

INSERT ALL

```
INTO Show VALUES (1, TO_TIMESTAMP('2025-10-10 19:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-10 22:00', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 30, 1, 1)
```

```
INTO Show VALUES (2, TO_TIMESTAMP('2025-10-10 20:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-10 22:00', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 20, 2, 2)
```

```
INTO Show VALUES (3, TO_TIMESTAMP('2025-10-11 18:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-11 20:30', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 15, 3, 3)
```

```
INTO Show VALUES (4, TO_TIMESTAMP('2025-10-12 16:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-12 22:00', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 25, 4, 4)
```

```
INTO Show VALUES (5, TO_TIMESTAMP('2025-10-12 19:30', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-12 21:10', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 20, 5, 5)
```

```
INTO Show VALUES (6, TO_TIMESTAMP('2025-10-13 18:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-13 19:30', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 10, 6, 6)
```

```
INTO Show VALUES (7, TO_TIMESTAMP('2025-10-13 20:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-13 22:10', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 15, 7, 7)
```

```
INTO Show VALUES (8, TO_TIMESTAMP('2025-10-14 17:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-14 22:00', 'YYYY-MM-DD HH24:MI'),
    'Scheduled', 25, 8, 8)
```

SELECT * FROM dual;

		START_TIME	END_TIME	STATUS	DOORS_OPEN_BEFO	EVENT_ID	HALL_ID
1	1	2025-10-10T19:00:00	2025-10-10T22:00:00	Scheduled	30	1	1
2	2	2025-10-10T20:00:00	2025-10-10T22:00:00	Scheduled	20	2	2
3	3	2025-10-11T18:00:00	2025-10-11T20:30:00	Scheduled	15	3	3
4	4	2025-10-12T16:00:00	2025-10-12T22:00:00	Scheduled	25	4	4
5	5	2025-10-12T19:30:00	2025-10-12T21:10:00	Scheduled	20	5	5
6	6	2025-10-13T18:00:00	2025-10-13T19:30:00	Scheduled	10	6	6
7	7	2025-10-13T20:00:00	2025-10-13T22:10:00	Scheduled	15	7	7
8	8	2025-10-14T17:00:00	2025-10-14T22:00:00	Scheduled	25	8	8

7. Seat

```
CREATE TABLE Seat (  
    seat_id INT PRIMARY KEY,  
    row_label CHAR(1) NOT NULL,  
    seat_no INT NOT NULL,  
    zone VARCHAR(20),  
    status VARCHAR(20) DEFAULT 'Available',  
    hall_id INT NOT NULL,  
    FOREIGN KEY (hall_id) REFERENCES Hall(hall_id)  
);
```

Result

Table SEAT created.

Elapsed: 00:00:00.021

Sample Records (8)

```
INSERT INTO Seat VALUES
(1, 'A', 1, 'Premium', 'Available', 1),
(2, 'A', 2, 'Premium', 'Available', 1),
(3, 'B', 1, 'Standard', 'Available', 1),
(4, 'B', 2, 'Standard', 'Available', 1),
(5, 'C', 1, 'Premium', 'Available', 2),
(6, 'C', 2, 'Premium', 'Available', 2),
(7, 'D', 1, 'Standard', 'Available', 2),
(8, 'D', 2, 'Standard', 'Available', 2);
```

	SEAT_ID	ROW_LABEL	SEAT_NO	ZONE	STATUS	HALL_ID
1		1 A		1 Premium	Available	1
2		2 A		2 Premium	Available	1
3		3 B		1 Standard	Available	1
4		4 B		2 Standard	Available	1
5		5 C		1 Premium	Available	2
6		6 C		2 Premium	Available	2
7		7 D		1 Standard	Available	2
8		8 D		2 Standard	Available	2

8. PricingPlan

```
CREATE TABLE PricingPlan (
  plan_id INT PRIMARY KEY,
  zone VARCHAR(20) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  convenience_fee DECIMAL(10,2),
  valid_from DATE,
  valid_to DATE,
  show_id INT NOT NULL,
  FOREIGN KEY (show_id) REFERENCES Show(show_id)
);
```

Result

Table PRICINGPLAN created.

Elapsed: 00:00:00.023

Sample Records (8)

INSERT ALL

```
INTO PricingPlan VALUES (1, 'Premium', 500, 20, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 1)  
INTO PricingPlan VALUES (2, 'Standard', 300, 15, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 1)  
INTO PricingPlan VALUES (3, 'VIP', 1000, 50, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 2)  
INTO PricingPlan VALUES (4, 'Premium', 600, 25, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 2)  
INTO PricingPlan VALUES (5, 'Standard', 350, 15, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 3)  
INTO PricingPlan VALUES (6, 'Premium', 550, 20, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 3)  
INTO PricingPlan VALUES (7, 'VIP', 1200, 60, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 4)  
INTO PricingPlan VALUES (8, 'Standard', 400, 20, TO_DATE('2025-10-01', 'YYYY-MM-DD'),  
    TO_DATE('2025-12-31', 'YYYY-MM-DD'), 4)
```

SELECT * FROM dual;

	PLAN_ID	ZONE	PRICE	CONVENIENCE_FEE	VALID_FROM	VALID_TO	SHOW_ID
1	1	Premium	500	20	10/1/2025, 12:00:00	12/31/2025, 12:00:00	1
2	2	Standard	300	15	10/1/2025, 12:00:00	12/31/2025, 12:00:00	1
3	3	VIP	1000	50	10/1/2025, 12:00:00	12/31/2025, 12:00:00	2
4	4	Premium	600	25	10/1/2025, 12:00:00	12/31/2025, 12:00:00	2
5	5	Standard	350	15	10/1/2025, 12:00:00	12/31/2025, 12:00:00	3
6	6	Premium	550	20	10/1/2025, 12:00:00	12/31/2025, 12:00:00	3
7	7	VIP	1200	60	10/1/2025, 12:00:00	12/31/2025, 12:00:00	4
8	8	Standard	400	20	10/1/2025, 12:00:00	12/31/2025, 12:00:00	4

9. CartHold

```
CREATE TABLE CartHold (  
    hold_id NUMBER PRIMARY KEY,  
    created_at TIMESTAMP NOT NULL,  
    expires_at TIMESTAMP NOT NULL,  
    status VARCHAR2(20),  
    patron_id NUMBER NOT NULL,  
    show_id NUMBER NOT NULL,  
    FOREIGN KEY (patron_id) REFERENCES Patron(patron_id),  
    FOREIGN KEY (show_id) REFERENCES Show(show_id),  
    CHECK (expires_at > created_at)  
);
```

Result

Table CARTHOLD created.

Elapsed: 00:00:00.021



Sample Records (8)

```
INSERT ALL
  INTO CartHold VALUES (1, TO_TIMESTAMP('2025-10-07 18:00', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:10', 'YYYY-MM-DD HH24:MI'),
    'Active', 1, 1)
  INTO CartHold VALUES (2, TO_TIMESTAMP('2025-10-07 18:05', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:15', 'YYYY-MM-DD HH24:MI'),
    'Expired', 2, 1)
  INTO CartHold VALUES (3, TO_TIMESTAMP('2025-10-07 18:10', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:20', 'YYYY-MM-DD HH24:MI'),
    'Active', 3, 2)
  INTO CartHold VALUES (4, TO_TIMESTAMP('2025-10-07 18:15', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:25', 'YYYY-MM-DD HH24:MI'),
    'Active', 4, 2)
  INTO CartHold VALUES (5, TO_TIMESTAMP('2025-10-07 18:20', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:30', 'YYYY-MM-DD HH24:MI'),
    'Expired', 5, 3)
  INTO CartHold VALUES (6, TO_TIMESTAMP('2025-10-07 18:25', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:35', 'YYYY-MM-DD HH24:MI'),
    'Active', 6, 3)
  INTO CartHold VALUES (7, TO_TIMESTAMP('2025-10-07 18:30', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:40', 'YYYY-MM-DD HH24:MI'),
    'Active', 7, 4)
  INTO CartHold VALUES (8, TO_TIMESTAMP('2025-10-07 18:35', 'YYYY-MM-DD HH24:MI'),
    TO_TIMESTAMP('2025-10-07 18:45', 'YYYY-MM-DD HH24:MI'),
    'Expired', 8, 4)
SELECT * FROM dual;
```

	HOLD_ID	CREATED_AT	EXPIRES_AT	STATUS	PATRON_ID	SHOW_ID	
1	1	2025-10-07T18:00:00	2025-10-07T18:10:00	Active	1	1	
2	2	2025-10-07T18:05:00	2025-10-07T18:15:00	Expired	2	1	
3	3	2025-10-07T18:10:00	2025-10-07T18:20:00	Active	3	2	
4	4	2025-10-07T18:15:00	2025-10-07T18:25:00	Active	4	2	
5	5	2025-10-07T18:20:00	2025-10-07T18:30:00	Expired	5	3	
6	6	2025-10-07T18:25:00	2025-10-07T18:35:00	Active	6	3	
7	7	2025-10-07T18:30:00	2025-10-07T18:40:00	Active	7	4	
8	8	2025-10-07T18:35:00	2025-10-07T18:45:00	Expired	8	4	

10. Booking

```
CREATE TABLE Booking (  
  booking_id NUMBER PRIMARY KEY,  
  total_amount NUMBER(10,2) NOT NULL,  
  booking_time TIMESTAMP NOT NULL,  
  status VARCHAR2(20),  
  patron_id NUMBER NOT NULL,  
  show_id NUMBER NOT NULL,  
  payment_id NUMBER,  
  FOREIGN KEY (patron_id) REFERENCES Patron(patron_id),  
  FOREIGN KEY (show_id) REFERENCES Show(show_id)  
);
```

Result

Table BOOKING created.

Elapsed: 00:00:00.030

Sample Records (8)

```
INSERT INTO Booking VALUES  
(1, 520, '2025-10-07 18:12', 'Confirmed', 1, 1, 1),  
(2, 310, '2025-10-07 18:15', 'Cancelled', 2, 1, 2),  
(3, 600, '2025-10-07 18:20', 'Confirmed', 3, 2, 3),  
(4, 625, '2025-10-07 18:25', 'Confirmed', 4, 2, 4),  
(5, 350, '2025-10-07 18:30', 'Confirmed', 5, 3, 5),  
(6, 575, '2025-10-07 18:35', 'Confirmed', 6, 3, 6),  
(7, 1220, '2025-10-07 18:40', 'Confirmed', 7, 4, 7),  
(8, 420, '2025-10-07 18:45', 'Confirmed', 8, 4, 8);
```

	BOOKING_ID	TOTAL_AMOUNT	BOOKING_TIME	STATUS	PATRON_ID	SHOW_ID	PAYMENT_ID
1	1	520	2025-10-07T18:12:00	Confirmed	1	1	1
2	2	310	2025-10-07T18:15:00	Cancelled	2	1	2
3	3	600	2025-10-07T18:20:00	Confirmed	3	2	3
4	4	625	2025-10-07T18:25:00	Confirmed	4	2	4
5	5	350	2025-10-07T18:30:00	Confirmed	5	3	5
6	6	575	2025-10-07T18:35:00	Confirmed	6	3	6
7	7	1220	2025-10-07T18:40:00	Confirmed	7	4	7
8	8	420	2025-10-07T18:45:00	Confirmed	8	4	8

11. Payment

```
CREATE TABLE Payment (  
  payment_id INT PRIMARY KEY,  
  amount DECIMAL(10,2) NOT NULL,  
  method VARCHAR(20) NOT NULL,  
  status VARCHAR(20),  
  gateway_ref VARCHAR(50),  
  booking_id INT NOT NULL,  
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
```

);

Table PAYMENT created.

Elapsed: 00:00:00.033



Sample Records (8)

```
INSERT INTO Payment VALUES
(1, 520, 'UPI', 'Success', 'UPI001', 1),
(2, 310, 'Card', 'Failed', 'CARD002', 2),
(3, 600, 'UPI', 'Success', 'UPI003', 3),
(4, 625, 'Wallet', 'Success', 'WAL004', 4),
(5, 350, 'Card', 'Success', 'CARD005', 5),
(6, 575, 'UPI', 'Success', 'UPI006', 6),
(7, 1220, 'Card', 'Success', 'CARD007', 7),
(8, 420, 'Wallet', 'Success', 'WAL008', 8);
```

Download ▾ Execution time: 0.014 seconds

	PAYMENT_ID	AMOUNT	METHOD	STATUS	GATEWAY_REF	BOOKING_ID
1	1	520	UPI	Success	UPI001	1
2	2	310	Card	Failed	CARD002	2
3	3	600	UPI	Success	UPI003	3
4	4	625	Wallet	Success	WAL004	4
5	5	350	Card	Success	CARD005	5
6	6	575	UPI	Success	UPI006	6
7	7	1220	Card	Success	CARD007	7
8	8	420	Wallet	Success	WAL008	8

12. Ticket

```
CREATE TABLE Ticket (
  ticket_id NUMBER PRIMARY KEY,
  qr_code VARCHAR2(50) NOT NULL UNIQUE,
  status VARCHAR2(20) DEFAULT 'Reserved',
  admitted_at TIMESTAMP,
  booking_id NUMBER NOT NULL,
  seat_id NUMBER NOT NULL,
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
  FOREIGN KEY (seat_id) REFERENCES Seat(seat_id)
);
```

Result

Table TICKET created.

Elapsed: 00:00:00.029



Sample Records (8)

INSERT ALL

INTO Ticket VALUES (1, 'QR001', 'Admitted', TO_TIMESTAMP('2025-10-07 19:00', 'YYYY-MM-DD HH24:MI'), 1, 1)

INTO Ticket VALUES (2, 'QR002', 'Reserved', NULL, 1, 2)

INTO Ticket VALUES (3, 'QR003', 'Cancelled', NULL, 2, 3)

INTO Ticket VALUES (4, 'QR004', 'Admitted', TO_TIMESTAMP('2025-10-07 19:15', 'YYYY-MM-DD HH24:MI'), 3, 4)

INTO Ticket VALUES (5, 'QR005', 'Admitted', TO_TIMESTAMP('2025-10-07 19:30', 'YYYY-MM-DD HH24:MI'), 4, 5)

INTO Ticket VALUES (6, 'QR006', 'Reserved', NULL, 5, 6)

INTO Ticket VALUES (7, 'QR007', 'Admitted', TO_TIMESTAMP('2025-10-07 20:00', 'YYYY-MM-DD HH24:MI'), 6, 7)

INTO Ticket VALUES (8, 'QR008', 'Reserved', NULL, 7, 8)

SELECT * FROM dual;

	TICKET_ID	QR_CODE	STATUS	ADMITTED_AT	BOOKING_ID	SEAT_ID
1	1	QR001	Admitted	2025-10-07T19:00:00	1	1
2	2	QR002	Reserved	(null)	1	2
3	3	QR003	Cancelled	(null)	2	3
4	4	QR004	Admitted	2025-10-07T19:15:00	3	4
5	5	QR005	Admitted	2025-10-07T19:30:00	4	5
6	6	QR006	Reserved	(null)	5	6
7	7	QR007	Admitted	2025-10-07T20:00:00	6	7
8	8	QR008	Reserved	(null)	7	8

13. Review

CREATE TABLE Review (

review_id INT PRIMARY KEY,

rating INT CHECK (rating BETWEEN 1 AND 5),

comment VARCHAR(255),

created_at DATETIME NOT NULL,

patron_id INT NOT NULL,

event_id INT NOT NULL,

FOREIGN KEY (patron_id) REFERENCES Patron(patron_id),

FOREIGN KEY (event_id) REFERENCES Event(event_id)

);

Result

Table REVIEW created.

Elapsed: 00:00:00.022



Sample Records (8)

INSERT ALL

INTO Review VALUES (1, 5, 'Great movie experience', TO_TIMESTAMP('2025-10-08 10:00', 'YYYY-MM-DD HH24:MI'), 1, 1)

INTO Review VALUES (2, 4, 'Good sound quality', TO_TIMESTAMP('2025-10-08 10:15', 'YYYY-MM-

```

DD HH24:MI'), 2, 1)
  INTO Review VALUES (3, 3, 'Average seating', TO_TIMESTAMP('2025-10-08 10:30', 'YYYY-MM-DD
HH24:MI'), 3, 2)
  INTO Review VALUES (4, 5, 'Loved the play', TO_TIMESTAMP('2025-10-08 11:00', 'YYYY-MM-DD
HH24:MI'), 4, 2)
  INTO Review VALUES (5, 2, 'Too crowded', TO_TIMESTAMP('2025-10-08 11:15', 'YYYY-MM-DD
HH24:MI'), 5, 3)
  INTO Review VALUES (6, 4, 'Comfortable seats', TO_TIMESTAMP('2025-10-08 11:30', 'YYYY-MM-DD
HH24:MI'), 6, 3)
  INTO Review VALUES (7, 5, 'Excellent visuals', TO_TIMESTAMP('2025-10-08 12:00', 'YYYY-MM-DD
HH24:MI'), 7, 4)
  INTO Review VALUES (8, 4, 'Good ambience', TO_TIMESTAMP('2025-10-08 12:15', 'YYYY-MM-DD
HH24:MI'), 8, 4)
SELECT * FROM dual;

```

	REVIEW_ID	RATING	COMMET	CREATED_AT	PATRON_ID	EVENT_ID	
1	1	5	Great movie experier	2025-10-08T10:00:00	1	1	
2	2	4	Good sound quality	2025-10-08T10:15:00	2	1	
3	3	3	Average seating	2025-10-08T10:30:00	3	2	
4	4	5	Loved the play	2025-10-08T11:00:00	4	2	
5	5	2	Too crowded	2025-10-08T11:15:00	5	3	
6	6	4	Comfortable seats	2025-10-08T11:30:00	6	3	
7	7	5	Excellent visuals	2025-10-08T12:00:00	7	4	
8	8	4	Good ambience	2025-10-08T12:15:00	8	4	

14. SupportCase

```

CREATE TABLE SupportCase (
  case_id INT PRIMARY KEY,
  issue_type VARCHAR(50),
  status VARCHAR(20),
  opened_at DATETIME NOT NULL,
  patron_id INT NOT NULL,
  booking_id INT NOT NULL,
  FOREIGN KEY (patron_id) REFERENCES Patron(patron_id),
  FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);

```

Result

Table SUPPORTCASE created.

Elapsed: 00:00:00.028



Sample Records (8)

```

INSERT INTO SupportCase VALUES
(1, 'Payment Failure', 'Resolved', '2025-10-07 18:20', 2, 2),
(2, 'Seat Not Reserved', 'Resolved', '2025-10-07 18:25', 5, 5),
(3, 'QR Not Scanning', 'Pending', '2025-10-07 19:05', 1, 1),
(4, 'Booking Cancellation', 'Resolved', '2025-10-07 19:10', 3, 2),

```

```
(5, 'Refund Delay', 'Pending', '2025-10-07 19:15', 6, 6),
(6, 'App Crash', 'Resolved', '2025-10-07 19:20', 4, 4),
(7, 'Wrong Seat Assigned', 'Pending', '2025-10-07 19:25', 7, 7),
(8, 'Duplicate Booking', 'Resolved', '2025-10-07 19:30', 8, 8);
```

	CASE_ID	ISSUE_TYPE	STATUS	OPENED_AT	PATRON_ID	BOOKING_ID
1	8	Duplicate Booking	Resolved	2025-10-07T19:30:00	8	8
2	1	Payment Failure	Resolved	2025-10-07T18:20:00	2	2
3	2	Seat Not Reserved	Resolved	2025-10-07T18:25:00	5	5
4	3	QR Not Scanning	Pending	2025-10-07T19:05:00	1	1
5	4	Booking Cancellation	Resolved	2025-10-07T19:10:00	3	2
6	5	Refund Delay	Pending	2025-10-07T19:15:00	6	6
7	6	App Crash	Resolved	2025-10-07T19:20:00	4	4
8	7	Wrong Seat Assigned	Pending	2025-10-07T19:25:00	7	7

15. LoyaltyWallet

```
CREATE TABLE LoyaltyWallet (
  wallet_id INT PRIMARY KEY,
  balance DECIMAL(10,2) DEFAULT 0,
  points INT DEFAULT 0,
  patron_id INT UNIQUE NOT NULL,
  FOREIGN KEY (patron_id) REFERENCES Patron(patron_id)
);
```

Result

Table LOYALTYWALLET created.

Elapsed: 00:00:00.027

Sample Records (8)

```
INSERT ALL
  INTO LoyaltyWallet VALUES (1, 100.0, 50, 1)
  INTO LoyaltyWallet VALUES (2, 50.0, 20, 2)
  INTO LoyaltyWallet VALUES (3, 200.0, 80, 3)
  INTO LoyaltyWallet VALUES (4, 150.0, 60, 4)
  INTO LoyaltyWallet VALUES (5, 75.0, 30, 5)
  INTO LoyaltyWallet VALUES (6, 120.0, 40, 6)
  INTO LoyaltyWallet VALUES (7, 250.0, 100, 7)
  INTO LoyaltyWallet VALUES (8, 90.0, 35, 8)
SELECT * FROM dual;
```

	WALLET_ID	BALANCE	POINTS	PATRON_ID
1	1	100	50	1
2	2	50	20	2
3	3	200	80	3
4	4	150	60	4
5	5	75	30	5
6	6	120	40	6
7	7	250	100	7
8	8	90	35	8

16. Companion

```
CREATE TABLE Companion (
  companion_id INT PRIMARY KEY,
  name VARCHAR(50),
  phone VARCHAR(15),
  patron_id INT NOT NULL,
  FOREIGN KEY (patron_id) REFERENCES Patron(patron_id)
);
```

Result

Table COMPANION created.

Elapsed: 00:00:00.034



Sample Records (8)

```
INSERT ALL
  INTO Companion VALUES (1, 'Meera', '9876543210', 1)
  INTO Companion VALUES (2, 'Rohit', '9876543211', 2)
  INTO Companion VALUES (3, 'Anjali', '9876543212', 3)
  INTO Companion VALUES (4, 'Kiran', '9876543213', 4)
  INTO Companion VALUES (5, 'Priya', '9876543214', 5)
  INTO Companion VALUES (6, 'Sandeep', '9876543215', 6)
  INTO Companion VALUES (7, 'Neha', '9876543216', 7)
  INTO Companion VALUES (8, 'Vikram', '9876543217', 8)
SELECT * FROM dual;
```

	COMPANION_ID	NAME	PHONE	PATRON_ID
1	1	Meera	9876543210	1
2	2	Rohit	9876543211	2
3	3	Anjali	9876543212	3
4	4	Kiran	9876543213	4
5	5	Priya	9876543214	5
6	6	Sandeep	9876543215	6
7	7	Neha	9876543216	7
8	8	Vikram	9876543217	8

5. Queries with Aggregate Operators

5.1 Query with MAX Operator

Find the maximum total booking amount made by any patron.

→SELECT MAX(total_amount) AS Max_Booking_Amount
FROM Booking;

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.148 seconds				
	MAX_BOOKING_AM			
1	1220			

5.2 Query with MIN Operator

Find the minimum price from all pricing plans.

→SELECT MIN(price) AS Min_Ticket_Price
FROM PricingPlan;

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.149 seconds				
	MIN_TICKET_PRICE			
1	300			

5.3 Query with AVG Operator

Find the average rating given by patrons in reviews.

```
→SELECT AVG(rating) AS Average_Rating  
FROM Review;
```

Output:

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.006 seconds				
	AVERAGE_RATING			
1	4			

5.4 Query with SUM Operator

Find the total payment collected for all bookings.

```
→SELECT SUM(amount) AS Total_Revenue  
FROM Payment;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.01 seconds				
	TOTAL_REVENUE			
1	4620			

5.5 Query with COUNT Operator

Find the total number of tickets sold.

```
→SELECT COUNT(ticket_id) AS Total_Tickets_Sold  
FROM Ticket;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.009 seconds				
	TOTAL_TICKETS_SOLD			
1	8			

Group By + Having Queries

5.6 MAX with GROUP BY & HAVING

Find the maximum booking amount for each patron who spent more than ₹2000.

```
→SELECT patron_id, MAX(total_amount) AS Max_Spent
FROM Booking
GROUP BY patron_id
HAVING MAX(total_amount) > 2000;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.007 seconds

PATRON_ID	MAX_SPENT
-----------	-----------

No items to display.

5.7 MIN with GROUP BY & HAVING

Find the minimum ticket price for each show where the lowest ticket is below ₹150.

```
→SELECT show_id, MIN(price) AS Min_Price
FROM PricingPlan
GROUP BY show_id
HAVING MIN(price) < 150;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download



Execution time: 0.008 seconds

SHOW_ID	MIN_PRICE
---------	-----------

No items to display.

5.8 AVG with GROUP BY & HAVING

Find average rating for each event with an average above 4.

```
→SELECT event_id, AVG(rating) AS Avg_Rating
FROM Review
GROUP BY event_id
HAVING AVG(rating) > 4;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.007 seconds				
	EVENT_ID	AVG_RATING		
1	1	4.5		
2	4	4.5		

5.9 SUM with GROUP BY & HAVING

Find total payments received for each booking where total exceeds ₹500.

```
→SELECT booking_id, SUM(amount) AS Total_Payment
FROM Payment
GROUP BY booking_id
HAVING SUM(amount) > 500;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.006 seconds				
	BOOKING_ID	TOTAL_PAYMENT		
1	1	520		
2	3	600		
3	4	625		
4	6	575		
5	7	1220		

5.10 COUNT with GROUP BY & HAVING

Find number of tickets booked for each show where more than 5 tickets were sold.

```
→SELECT s.show_id, COUNT(t.ticket_id) AS Tickets_Booked
FROM Ticket t
JOIN Booking b ON t.booking_id = b.booking_id
JOIN Show s ON b.show_id = s.show_id
GROUP BY s.show_id
HAVING COUNT(t.ticket_id) > 5;
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

▼

Execution time: 0.017 seconds

SHOW_ID

TICKETS_BOOKED

No items to display.

6. Nested Queries

6.1 Query using IN operator

Find all patrons who have made bookings for shows that belong to events organized by “PVR Cinemas”.

```
→SELECT patron_id, name
FROM Patron
WHERE patron_id IN (
  SELECT DISTINCT b.patron_id
  FROM Booking b
  JOIN Show s ON b.show_id = s.show_id
  JOIN Event e ON s.event_id = e.event_id
  JOIN Organizer o ON e.organizer_id = o.org_id
  WHERE o.name = 'PVR Cinemas'
);
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download 

Execution time: 0.034 seconds

PATRON_ID	NAME
No items to display.	

6.2 Query using NOT IN operator

Find all patrons who have never written a review for any event.

```
→SELECT patron_id, name
FROM Patron
WHERE patron_id NOT IN (
  SELECT DISTINCT patron_id
  FROM Review
);
```

Query result

Script output

DBMS output

Explain Plan

SQL history





Download 

Execution time: 0.034 seconds

PATRON_ID	NAME
No items to display.	

6.3 Query using EXISTS operator

Find events that have at least one review.

```
→SELECT e.event_id, e.title
FROM Event e
```

```
WHERE EXISTS (
  SELECT 1
  FROM Review r
  WHERE r.event_id = e.event_id
);
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.009 seconds				
	EVENT_ID	TITLE		
1	1	Avengers: Endgame		
2	2	Rock Concert		
3	3	Shakespeare Play		
4	4	Cricket Match		

6.4 Query using NOT EXISTS operator

Find all shows that have no bookings yet.

```
→SELECT s.show_id, s.start_time, s.end_time
FROM Show s
WHERE NOT EXISTS (
  SELECT 1
  FROM Booking b
  WHERE b.show_id = s.show_id
);
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▼ Execution time: 0.011 seconds				
	SHOW_ID	START_TIME	END_TIME	
1	6	2025-10-13T18:00:00Z	2025-10-13T19:30:00Z	
2	7	2025-10-13T20:00:00Z	2025-10-13T22:10:00Z	
3	8	2025-10-14T17:00:00Z	2025-10-14T22:00:00Z	
4	5	2025-10-12T19:30:00Z	2025-10-12T21:10:00Z	

6.5 Query using ANY operator

Find all bookings whose total amount is greater than the average of any payment made by patrons.

```

→SELECT booking_id, total_amount
FROM Booking
WHERE total_amount > ANY (
    SELECT amount
    FROM Payment
);

```

Query result	Script output	DBMS output	Explain Plan
  Download ▼ Execution time: 0.01 seconds			
	BOOKING_ID	TOTAL_AMOUNT	
1	7	1220	
2	4	625	
3	3	600	
4	6	575	
5	1	520	
6	8	420	
7	5	350	

6.6 Query using ALL operator

Find bookings whose total amount is greater than all payment amounts made (highest payment).

```

→SELECT booking_id, total_amount
FROM Booking
WHERE total_amount > ALL (
    SELECT amount
    FROM Payment
);

```

Query result	Script output	DBMS output	Exp
  Download ▼ Execution time: 0.005 sec			
BOOKING_ID	TOTAL_AMOUNT		
No items to display.			

7.SET OPERATORS

7.1 Query using UNION operator

Objective: Find all patrons who have made bookings or written reviews (combined list, no duplicates).

```

SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Booking
)
UNION

```

```

SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Review
);

```

Query result	Script output	DBMS output	Expl
  Download ▼ Execution time: 0.016 sec			
	PATRON_ID	NAME	
1	1	Arun	
2	2	Meera	
3	3	Ravi	
4	4	Priya	
5	5	Anil	
6	6	Sara	
7	7	Vikram	
8	8	Neha	

7.2 Query using INTERSECT operator

Objective: Find patrons who have both made bookings and written reviews.

```

SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Booking
)
INTERSECT
SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Review
);

```


Query result	Script output	DBMS output	Explain
  Download ▼ Execution time: 0.011 second			
	PATRON_ID	NAME	
1	1	Arun	
2	2	Meera	
3	3	Ravi	
4	4	Priya	
5	5	Anil	
6	6	Sara	
7	7	Vikram	
8	8	Neha	

7.3 Query using EXCEPT operator

Objective: Find patrons who have made bookings but never written a review.

```
SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Booking
)
EXCEPT
SELECT DISTINCT patron_id, name
FROM Patron
WHERE patron_id IN (
    SELECT patron_id
    FROM Review
);
```

Query result	Script output	DBMS output
  Download ▼ Execution time: 0.		
PATRON_ID	NAME	
No items to display.		

8. JOINS

8.1 Query using INNER JOIN

Objective: List all bookings along with patron names and show timings.

```
SELECT b.booking_id, p.name AS patron_name, s.start_time, s.end_time
FROM Booking b
INNER JOIN Patron p ON b.patron_id = p.patron_id
INNER JOIN Show s ON b.show_id = s.show_id;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
Download ▾ Execution time: 0.012 seconds				
	BOOKING_ID	PATRON_NAME	START_TIME	END_TIME
1	3	Ravi	2025-10-10T20:00:00Z	2025-10-10T22:00:00Z
2	4	Priya	2025-10-10T20:00:00Z	2025-10-10T22:00:00Z
3	5	Anil	2025-10-11T18:00:00Z	2025-10-11T20:30:00Z
4	6	Sara	2025-10-11T18:00:00Z	2025-10-11T20:30:00Z
5	7	Vikram	2025-10-12T16:00:00Z	2025-10-12T22:00:00Z
6	8	Neha	2025-10-12T16:00:00Z	2025-10-12T22:00:00Z
7	1	Arun	2025-10-10T19:00:00Z	2025-10-10T22:00:00Z
8	2	Meera	2025-10-10T19:00:00Z	2025-10-10T22:00:00Z

8.2 Query using LEFT JOIN

Objective: List all patrons and their bookings (if any).

```
SELECT p.patron_id, p.name, b.booking_id, b.total_amount
FROM Patron p
LEFT JOIN Booking b ON p.patron_id = b.patron_id;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
Download ▾ Execution time: 0.005 seconds				
	PATRON_ID	NAME	BOOKING_ID	TOTAL_AMOUNT
1	1	Arun	1	520
2	2	Meera	2	310
3	3	Ravi	3	600
4	4	Priya	4	625
5	5	Anil	5	350
6	6	Sara	6	575
7	7	Vikram	7	1220
8	8	Neha	8	420

8.3 Query using RIGHT JOIN

Objective: List all bookings and corresponding patrons.

```
SELECT b.booking_id, b.total_amount, p.name AS patron_name
FROM Booking b
RIGHT JOIN Patron p ON b.patron_id = p.patron_id;
```

Query result	Script output	DBMS output	Explain Plan	SQL his
Download ▾ Execution time: 0.01 seconds				
	BOOKING_ID	TOTAL_AMOUNT	PATRON_NAME	
1	1	520	Arun	
2	2	310	Meera	
3	3	600	Ravi	
4	4	625	Priya	
5	5	350	Anil	
6	6	575	Sara	
7	7	1220	Vikram	
8	8	420	Neha	

8.4 Query using FULL JOIN

Objective: List all patrons and bookings, including patrons without bookings and bookings without patrons.

```
SELECT p.patron_id, p.name, b.booking_id, b.total_amount
```

```
FROM Patron p
FULL OUTER JOIN Booking b ON p.patron_id = b.patron_id;
```

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download ▾ Execution time: 0.004 seconds				
	PATRON_ID	NAME	BOOKING_ID	TOTAL_AMOUNT
1	1	Arun	1	520
2	2	Meera	2	310
3	3	Ravi	3	600
4	4	Priya	4	625
5	5	Anil	5	350
6	6	Sara	6	575
7	7	Vikram	7	1220
8	8	Neha	8	420

9. VIEWS

Objective: *Demonstrate data independence, security, and simplified data access.*

View 1: Patrons with Active Bookings

```
CREATE VIEW Active_Bookings AS
SELECT p.patron_id, p.name, b.booking_id, b.total_amount, b.status
FROM Patron p
JOIN Booking b ON p.patron_id = b.patron_id
WHERE b.status = 'Confirmed';
```

Result

View ACTIVE_BOOKINGS created.

Elapsed: 00:00:00.012



View 2: Event Reviews Summary

```
CREATE VIEW Event_Review_Summary AS
SELECT e.event_id, e.title, COUNT(r.review_id) AS review_count, AVG(r.rating) AS avg_rating
FROM Event e
LEFT JOIN Review r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title;
```

Result

View EVENT_REVIEW_SUMMARY created.

Elapsed: 00:00:00.011



View 3: Patrons and Loyalty Points

```
CREATE VIEW Patron_Loyalty AS
SELECT p.patron_id, p.name, lw.balance, lw.points
```

```
FROM Patron p
JOIN LoyaltyWallet lw ON p.patron_id = lw.patron_id;
```

View 4: Show Seating Availability

```
CREATE VIEW Show_Seats AS
SELECT s.show_id, s.start_time, s.end_time, se.zone, COUNT(se.seat_id) AS available_seats
FROM Show s
JOIN Seat se ON se.hall_id = s.hall_id
WHERE se.status = 'Available'
GROUP BY s.show_id, s.start_time, s.end_time, se.zone;
```

Result

```
View SHOW_SEATS created.
```

```
Elapsed: 00:00:00.010
```



View 5: Ticket Status Overview

```
CREATE VIEW Ticket_Status AS
SELECT t.ticket_id, t.status, b.booking_id, p.name AS patron_name
FROM Ticket t
JOIN Booking b ON t.booking_id = b.booking_id
JOIN Patron p ON b.patron_id = p.patron_id;
```

Result

```
View TICKET_STATUS created.
```

```
Elapsed: 00:00:00.012
```



View 6: Payments Summary per Booking

```
CREATE VIEW Booking_Payments AS
SELECT b.booking_id, p.method, p.amount, p.status
FROM Booking b
JOIN Payment p ON b.booking_id = p.booking_id;
```

```
SQL> CREATE VIEW Booking_Payments AS
      SELECT b.booking_id, p.method, p.amount, p.status
      FROM Booking b
      JOIN Payment p ON b.booking_id = p.booking_id
```

View BOOKING_PAYMENTS created.

Elapsed: 00:00:00.013

View 7: Events by Organizer

```
CREATE VIEW Organizer_Events AS
SELECT o.org_id, o.name AS organizer_name, e.event_id, e.title
FROM Organizer o
JOIN Event e ON o.org_id = e.organizer_id;
```

```
SQL> CREATE VIEW Organizer_Events AS
      SELECT o.org_id, o.name AS organizer_name, e.event_id, e.title
      FROM Organizer o
      JOIN Event e ON o.org_id = e.organizer_id
```

View ORGANIZER_EVENTS created.

Elapsed: 00:00:00.013

View 8: Patrons with Companions

```
CREATE VIEW Patron_Companions AS
SELECT p.patron_id, p.name, c.companion_id, c.name AS companion_name
FROM Patron p
LEFT JOIN Companion c ON p.patron_id = c.patron_id;
```

```
SQL> CREATE VIEW Patron_Companions AS
      SELECT p.patron_id, p.name, c.companion_id, c.name AS companion_name
      FROM Patron p
      LEFT JOIN Companion c ON p.patron_id = c.patron_id
```

View PATRON_COMPANIONS created.

Elapsed: 00:00:00.013

View 9: Shows without Bookings

```
CREATE VIEW Shows_No_Booking AS
SELECT s.show_id, s.start_time, s.end_time
FROM Show s
LEFT JOIN Booking b ON s.show_id = b.show_id
WHERE b.booking_id IS NULL;
```

Result

```
View SHOWS_NO_BOOKING created.
```

```
Elapsed: 00:00:00.013
```



View 10: Pricing Plans per Show

```
CREATE VIEW Show_Pricing AS
SELECT s.show_id, s.start_time, s.end_time, pp.zone, pp.price, pp.convenience_fee
FROM Show s
JOIN PricingPlan pp ON s.show_id = pp.show_id;

SQL> CREATE VIEW Show_Pricing AS
      SELECT s.show_id, s.start_time, s.end_time, pp.zone, pp.price, pp.convenience_fee
      FROM Show s
      JOIN PricingPlan pp ON s.show_id = pp.show_id
```

```
View SHOW_PRICING created.
```

```
Elapsed: 00:00:00.010
```

10. PLSQL

10.1 Procedure

Procedure to Add New Booking:

```
CREATE OR REPLACE PROCEDURE Add_Booking(
  p_patron_id IN INT,
  p_show_id IN INT,
  p_total_amount IN DECIMAL
)
AS
BEGIN
  INSERT INTO Booking(booking_id, patron_id, show_id, total_amount, booking_time, status)
  VALUES (Booking_seq.NEXTVAL, p_patron_id, p_show_id, p_total_amount, SYSDATE, 'Confirmed');
END;
```

Result

Procedure ADD_BOOKING compiled

LINE/COL ERROR

8/5 PL/SQL: SQL Statement ignored
9/13 PL/SQL: ORA-02289: sequence does not exist
Errors: check compiler log
Elapsed: 00:00:00.014

10.2 Cursor

Cursor to Fetch Bookings of a Patron:

```
DECLARE
  CURSOR booking_cursor IS
    SELECT booking_id, total_amount, status
    FROM Booking
    WHERE patron_id = 1;
  b_record booking_cursor%ROWTYPE;
BEGIN
  OPEN booking_cursor;
  LOOP
    FETCH booking_cursor INTO b_record;
    EXIT WHEN booking_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Booking ID: ' || b_record.booking_id || ', Amount: ' ||
b_record.total_amount);
  END LOOP;
  CLOSE booking_cursor;
END;
```

Result

Booking ID: 1, Amount: 520

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.013

10.3 Triggers

10.3.1 AFTER TRIGGER

```
CREATE OR REPLACE TRIGGER after_booking
AFTER INSERT ON Booking
```

```
FOR EACH ROW
BEGIN
    UPDATE LoyaltyWallet
    SET points = points + 10
    WHERE patron_id = :NEW.patron_id;
END;
/
```

Result

Trigger AFTER_BOOKING compiled

Elapsed: 00:00:00.020



10.3.2 BEFORE TRIGGER

```
Ensure total_amount is not negative
CREATE OR REPLACE TRIGGER before_booking
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    IF :NEW.total_amount < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Total amount cannot be negative');
    END IF;
END;
/
```

Result

Trigger BEFORE_BOOKING compiled

Elapsed: 00:00:00.024



10.3.3 ROW-LEVEL TRIGGER

```
Log each ticket insertion
CREATE OR REPLACE TRIGGER log_ticket_insert
AFTER INSERT ON Ticket
FOR EACH ROW
BEGIN
    INSERT INTO Ticket_Log(ticket_id, action, action_time)
    VALUES(:NEW.ticket_id, 'INSERT', SYSDATE);
END;
```


Result

Trigger LOG_TICKET_INSERT compiled

LINE/COL ERROR

2/5 PL/SQL: SQL Statement ignored
2/17 PL/SQL: ORA-00942: table or view does not exist
Errors: check compiler log
Elapsed: 00:00:00.016

10.3.4 STATEMENT-LEVEL TRIGGER

Track daily bookings count

```
CREATE OR REPLACE TRIGGER daily_booking_count
AFTER INSERT ON Booking
BEGIN
    UPDATE Booking_Stats
    SET daily_count = daily_count + 1
    WHERE stat_date = TRUNC(SYSDATE);
END;
```

Result

Trigger DAILY_BOOKING_COUNT compiled

LINE/COL ERROR

2/5 PL/SQL: SQL Statement ignored
2/12 PL/SQL: ORA-00942: table or view does not exist
Errors: check compiler log
Elapsed: 00:00:00.016

10.3.5 SET-ORIENTED TRIGGER

Apply discount to all bookings in a show after promotion

```
CREATE OR REPLACE TRIGGER apply_discount
AFTER UPDATE OF promotion_flag ON Show
BEGIN
    UPDATE Booking
    SET total_amount = total_amount * 0.9
    WHERE show_id IN (SELECT show_id FROM Show WHERE promotion_flag = 1);
END;
```

Result

Trigger DAILY_BOOKING_COUNT compiled

LINE/COL ERROR

2/5 PL/SQL: SQL Statement ignored
2/12 PL/SQL: ORA-00942: table or view does not exist
Errors: check compiler log
Elapsed: 00:00:00.014



11. Conclusion

- The BookMyShow database model supports complete booking lifecycle: patrons, shows, tickets, payments, and reviews.
- Views enhance data independence and security by restricting direct access to base tables.
- PLSQL procedures, cursors, and triggers automate business rules, enforce constraints, and maintain data integrity.
- Set operators, joins, and nested queries demonstrate flexible data retrieval and analytics.
- Overall, the DBMS design ensures efficiency, scalability, and reliable management of complex event-booking operations.

My Contributions

- Designed ER Diagram.
- Created relational schema.
- Developed database tables with PK/FK constraints.
- Implemented SQL queries (Joins, Aggregate, Nested Queries, Set Operators).
- Tested database using sample data.